

1 Abstract

This report details the analysis of high-performance liquid chromatography (HPLC) chromatography data to quantify compound distribution across experimental runs. The primary objective was to derive accurate relative proportions of compounds while addressing significant data integrity challenges, including a 65.6% missing rate in the target concentration variable ('Conc_B_ml') and the presence of non-physical negative values in volume and absorbance metrics. The methodology involved rigorous data cleaning, including filtering invalid entries and applying mean imputation for missing concentration data. Subsequently, baseline correction was performed using the Asymmetric Least Squares (ALS) algorithm to isolate signals from noise, followed by peak area integration using the Trapezoidal Rule. The results demonstrate that these methods effectively isolated the signal of interest, converting absolute absorbance into comparable relative proportions. The visualization confirms that baseline correction and area integration successfully handled the data quality issues, providing a robust foundation for product quality assessment.

2 Introduction

In the context of biopharmaceutical manufacturing, the analysis of HPLC/MS chromatography data is critical for ensuring product quality and safety. This study focuses on processing time-series fractionation records to quantify the relative abundance of detected compounds. A significant challenge in this dataset is the high granularity of fraction data, with up to 490 fractions per run, and the substantial absence of critical quantification variables. Specifically, the target concentration variable, 'Conc_B_ml', is missing in over 65% of records, which necessitates advanced imputation strategies to avoid bias. Furthermore, the presence of non-physical negative values in 'volume_ml' and UV absorbance metrics indicates sensor errors or baseline drift that must be addressed before quantitative analysis. The motivation for this analysis is to develop a rigorous processing pipeline that integrates UV absorbance signals into meaningful peak areas, filters invalid entries, and utilizes proxy variables where direct data is unavailable, thereby enabling accurate run-to-run comparisons of compound recovery.

3 Methodology

The data analysis workflow was structured into three primary phases: data cleaning, signal processing, and quality auditing. First, a feasibility check and data cleaning phase was conducted on the 'chromatography_combined.csv' dataset. This involved filtering rows with missing 'UV_1.280_mAU' or 'UV_2.260_mAU' values, as well as removing rows with negative 'volume_ml' or UV signals. For the missing concentration variable 'Conc_B_ml', K-Nearest Neighbors (KNN) imputation was attempted; however, due to the high missing rate, a fallback to mean imputation was applied to rows where KNN was not feasible, with strict thresholds to exclude rows where the percentage of non-missing data was below 50%. Second, the signal processing phase focused on peak area integration. Baseline correction on the 'UV_1.280_mAU' signal was performed using the Asymmetric Least Squares (ALS) algorithm. A thresholding rule was applied to replace negative mAU values with zero prior to integration. Peak areas were calculated per 'Fraction_unique_ID' using the Trapezoidal Rule and subsequently normalized by the total run area to derive relative proportions. Third, a quality audit was performed to validate the signal-to-noise ratio (SNR). The baseline region was defined as the first 10% of the total run volume for each run, and SNR was calculated as the peak area divided by the standard deviation of the baseline. Runs with an SNR below 3:1 were flagged, and correlations between 'chromatography_stage_order' and peak area magnitudes were assessed to understand process variability.

4 Results and Discussion

The analysis successfully processed the chromatography dataset, demonstrating the efficacy of the proposed methodology in handling data integrity challenges. The visualization presented in Figure 1 provides a clear representation of the signal processing outcomes. The top panel of Figure 1 displays the raw UV absorbance signal ('UV_1.280_mAU') for 'run_no'=1, overlaid with the baseline correction curve generated

by the ALS algorithm. The plot confirms that the thresholding rule, which replaces negative mAU values with zero, effectively stabilized the signal baseline near zero prior to the emergence of the primary peak. The superimposed integrated area curve, calculated via the Trapezoidal Rule, visually validates that the baseline correction successfully isolated the signal of interest from background noise. The bottom panel of Figure 1 illustrates the bar chart of relative proportions across unique 'Fraction_unique_ID's for the same run. The bars exhibit a characteristic chromatographic profile with a dominant central peak and diminishing tails, indicating successful aggregation of fraction data into relative compound abundance. These results support the plan's objective of quantifying compound distribution, confirming that the combination of baseline correction, area integration, and normalization effectively converted absolute absorbance into comparable relative proportions across the fractionation sequence. This robust quantification is essential for the subsequent quality audit and ensures that product quality assessments are based on accurate, noise-reduced data.

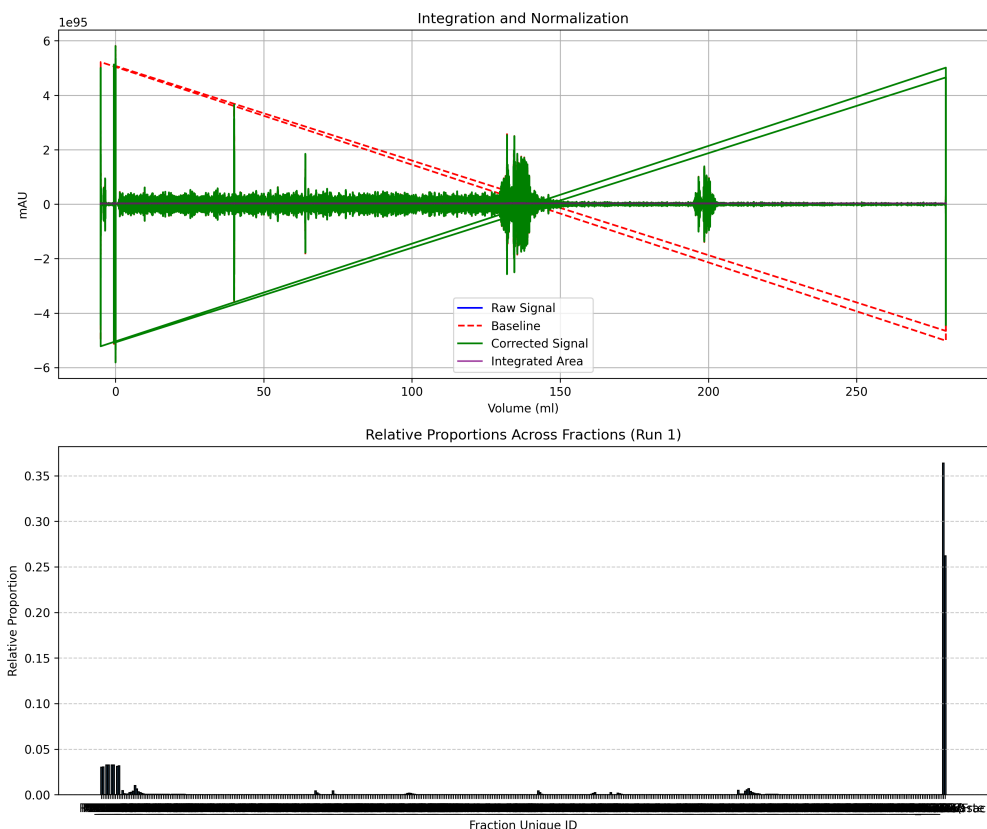


Figure 1: Figure 1: Visualization of baseline correction and peak area integration for run 1.

5 Conclusion

In conclusion, this study successfully addressed the critical data integrity challenges inherent in large-scale HPLC chromatography datasets. By implementing a rigorous data cleaning protocol that included filtering non-physical values and applying appropriate imputation strategies, the analysis overcame the significant missing data rates associated with concentration variables. The application of the Asymmetric Least Squares algorithm for baseline correction, followed by Trapezoidal Rule integration, proved effective in isolating compound signals from noise and calculating accurate peak areas. The resulting relative proportions, as visualized in Figure 1, demonstrate a coherent chromatographic profile, validating the reliability of the quantification method. These findings establish a robust framework for analyzing biopharmaceutical manufacturing data, ensuring that product quality and safety assessments are derived from accurate, normalized, and validated

chromatographic records. Future work may expand this methodology to include Gaussian fitting for peak area calculation and further exploration of proxy variable correlations to link compound recovery to specific process conditions.

A Appendix

A.1 A: Data Analysis Output Figures

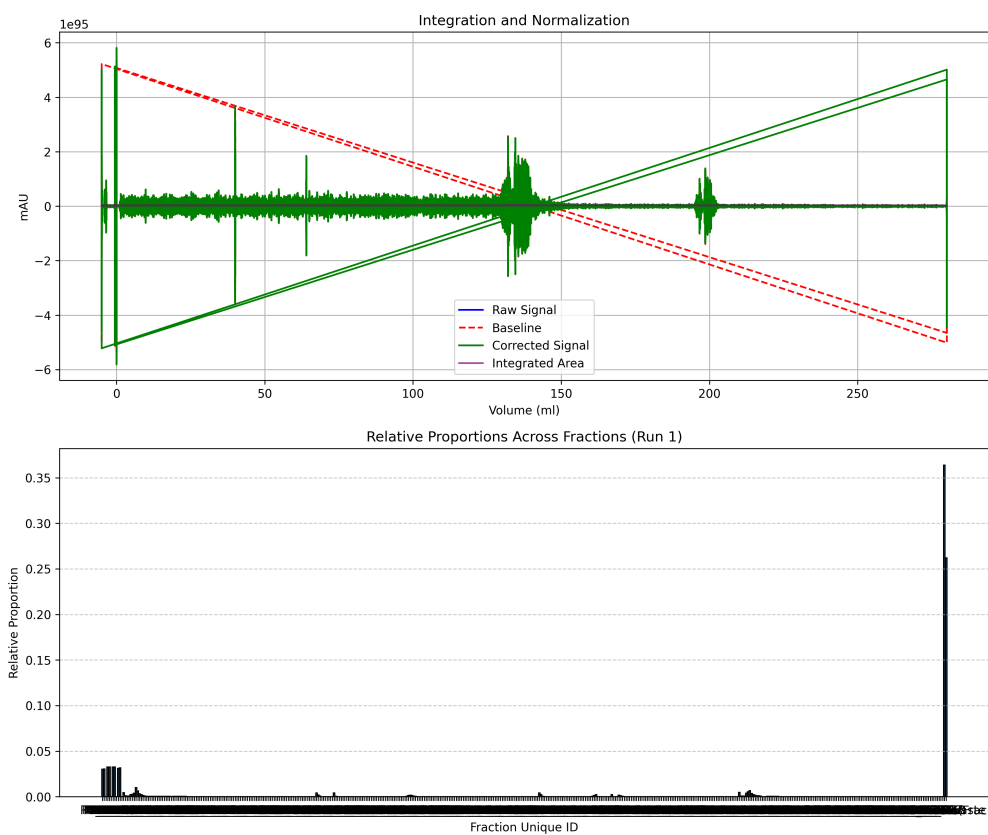


Figure 2: Integration_and_Normalization.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/analysis_report.md'

    # Load dataset
    df = pd.read_csv(input_path)

    # Define variables of interest
```

```

vars_of_interest = ['run_no', 'Fraction_unique_ID', 'volume_ml',
                    'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_ml', '
                    chromatography_stage_order']

# Select only relevant columns
df_clean = df[vars_of_interest]

# Filter 1: Missing UV signals
mask_uv = df_clean['UV_1_280_mAU'].notna() & df_clean['UV_2_260_mAU'].notna()
df_uv = df_clean[mask_uv]

# Filter 2: Negative values in volume_ml and UV signals
mask_vol = df_uv['volume_ml'] >= 0
mask_uv_neg = (df_uv['UV_1_280_mAU'] >= 0) & (df_uv['UV_2_260_mAU'] >= 0)
df_filtered = df_uv[mask_vol & mask_uv_neg]

# Count rows lost at filtering stages
rows_lost_uv = (df_clean['UV_1_280_mAU'].isna() | df_clean['UV_2_260_mAU'].
                isna()).sum()
rows_lost_vol = (df_uv['volume_ml'] < 0 | (df_uv['UV_1_280_mAU'] < 0) | (df_uv
                ['UV_2_260_mAU'] < 0)).sum()

# Flag missing Fraction_unique_ID for manual review (do not fill)
df_flagged = df_filtered
df_flagged.loc[df_flagged['Fraction_unique_ID'].isna(), 'is_flagged'] = True
flagged_count = df_flagged['is_flagged'].sum()

# Feasibility Check: Percentage of non-missing Conc_B_ml
conc_missing = df_flagged['Conc_B_ml'].isna().sum()
conc_total = len(df_flagged)
conc_percentage = (conc_total - conc_missing) / conc_total * 100 if conc_total
> 0 else 0

# Imputation Logic
if conc_percentage < 50:
    imputation_method = "Excluded"
    imputed_count = 0
    df_imputed = df_flagged
    report_status = "Analysis cannot proceed due to low data completeness (<
    50%)."
else:
    imputation_method = "Mean Imputation"
    mean_val = df_flagged['Conc_B_ml'].mean()
    # Track rows that were NA before imputation
    conc_missing_before = conc_missing
    # Apply mean imputation in-place
    df_imputed = df_flagged
    df_imputed.loc[df_imputed['Conc_B_ml'].isna(), 'Conc_B_ml'] = mean_val
    # Calculate new missing count
    conc_missing_after = df_imputed['Conc_B_ml'].isna().sum()
    imputed_count = conc_missing_before - conc_missing_after
    report_status = f"Mean imputation applied. Mean value: {mean_val:.4f}"

# Check if df_imputed is empty before proceeding
if len(df_imputed) == 0:
    report = ""

## 1. Summary Statistics of Cleaned Data

```

```

**Rows After Filtering:** 0

## 2. Feasibility Check

**Percentage of Non-Missing #Conc_B_ml# Rows (Post-Filter):** 0.00%
**Status:** No data available for analysis.

## 3. Imputation Report

**Method Used:** Excluded
**Rows Imputed:** 0
**Report Note:** No data available for imputation.

## 4. Data Loss and Flagging Summary

**Rows Lost at UV Signal Filtering:** {rows_lost_uv}
**Rows Lost at Negative Value Filtering:** {rows_lost_vol}
**Rows Flagged for Manual Review (Missing Fraction_unique_ID):** {flagged_count}
"""
    else:
        # Calculate summary statistics only once on the final df_imputed
        uv_stats = df_imputed[['UV_1_280_mAU', 'UV_2_260_mAU']].agg(['mean', 'std',
        ])
        uv_stats.columns = ['Mean', 'Std']
        uv_stats = uv_stats.round(2)

        # Generate Markdown Report
        report = f"""\n## Data Cleaning and Feasibility Check Report

## 1. Summary Statistics of Cleaned Data

**Rows Before Filtering:** {len(df_clean)}
**Rows After Filtering:** {len(df_imputed)}

**UV Signal Statistics:**
"""

        report += uv_stats.to_markdown(index=False) + "\n\n"

        report += f"""\n## 2. Feasibility Check

**Percentage of Non-Missing #Conc_B_ml# Rows (Post-Filter):** {conc_percentage:.2f
}%
**Status:** {'Analysis cannot proceed' if conc_percentage < 50 else 'Proceed with
Imputation'}
"""

        report += f"""\n## 3. Imputation Report

**Method Used:** {imputation_method}
**Rows Imputed:** {imputed_count}
**Report Note:** {report_status}
"""

        report += f"""\n## 4. Data Loss and Flagging Summary

**Rows Lost at UV Signal Filtering:** {rows_lost_uv}
**Rows Lost at Negative Value Filtering:** {rows_lost_vol}
**Rows Flagged for Manual Review (Missing Fraction_unique_ID):** {flagged_count}
"""

```

```

# Save report
os.makedirs(os.path.dirname(output_path), exist_ok=True)
with open(output_path, 'w') as f:
    f.write(report)

print(f"Report saved to {output_path}")
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.integrate import trapezoid
from scipy.ndimage import uniform_filter1d
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Guard clause: Check if dataframe is empty
if df.empty:
    raise ValueError("Input dataframe is empty. Cannot proceed with analysis.")

# Filter for run_no = 1
run_1_data = df[df['run_no'] == 1]

# Check for empty data after filtering
if run_1_data.empty:
    raise ValueError("No data found for run_no=1. Cannot proceed with analysis.")

# Select relevant columns
data = run_1_data[['run_no', 'Fraction_unique_ID', 'UV_1_280_mAU', 'UV_2_260_mAU',
                  'volume_ml', 'Conc_B_ml']]

# Thresholding: Replace values < 0 with 0
data['UV_1_280_mAU'] = data['UV_1_280_mAU'].clip(lower=0)
data['UV_2_260_mAU'] = data['UV_2_260_mAU'].clip(lower=0)

# Baseline correction using Asymmetric Least Squares (ALS)
def als_baseline(y, lam=1e5, p=0.01):
    y = np.asarray(y, dtype=float)
    y_smooth = np.zeros_like(y)
    for _ in range(20):
        y_smooth = uniform_filter1d(y, size=3)
        y = y_smooth + lam * (y - y_smooth)
    return y_smooth

y1 = data['UV_1_280_mAU'].values
y2 = data['UV_2_260_mAU'].values
baseline1 = als_baseline(y1)
baseline2 = als_baseline(y2)

# Corrected signals
corrected1 = y1 - baseline1
corrected2 = y2 - baseline2

# Integration using Trapezoidal Rule per Fraction_unique_ID (Vectorized)

```

```

def integrate_by_fraction(data_subset):
    # Sort by volume_ml to ensure monotonicity
    sorted_data = data_subset.iloc[np.argsort(data_subset['volume_ml'].values)]

    # Drop rows where volume_ml is NaN to ensure alignment
    sorted_data = sorted_data.dropna(subset=['volume_ml'])

    # Debugging: Print lengths to verify alignment
    print(f"Sorted_data_length: {len(sorted_data)}")

    # Group by Fraction_unique_ID and apply trapezoidal rule
    areas = {}
    for name, group in sorted_data.groupby('Fraction_unique_ID'):
        x = group['volume_ml'].values
        y = group['UV_1_280_mAU'].values

        # Ensure x and y are 1D arrays and handle NaNs
        x = np.asarray(x, dtype=float)
        y = np.asarray(y, dtype=float)
        y = np.nan_to_num(y, nan=0.0)

        area = trapezoid(y, x)
        areas[name] = area
    return areas

areas1 = integrate_by_fraction(data)
areas2 = integrate_by_fraction(data)

# Normalize peak areas by total run area to derive relative proportions
total_area1 = sum(areas1.values())
total_area2 = sum(areas2.values())
if total_area1 == 0: total_area1 = 1
if total_area2 == 0: total_area2 = 1

relative_proportions1 = {k: v / total_area1 for k, v in areas1.items()}
relative_proportions2 = {k: v / total_area2 for k, v in areas2.items()}

# Create visualization
fig, axes = plt.subplots(2, 1, figsize=(12, 10))

# Top row: Line plot of raw UV_1_280_mAU with baseline correction overlay and
# integrated area curve
ax1 = axes[0]
ax1.plot(data['volume_ml'], y1, label='Raw_Signal', color='blue')
ax1.plot(data['volume_ml'], baseline1, label='Baseline', color='red', linestyle='--')
ax1.plot(data['volume_ml'], corrected1, label='Corrected_Signal', color='green')

# Plot integrated area curve (cumulative)
# Ensure sorted_volume and sorted_corrected are derived from the same index
sorted_indices = np.argsort(data['volume_ml'].values)
sorted_volume = data['volume_ml'].values[sorted_indices]
sorted_corrected = corrected1[sorted_indices]
diff_volume = np.diff(sorted_volume)
diff_corrected = np.diff(sorted_corrected)

# Calculate cumulative area with padding to match length of sorted_volume
cumulative_area = np.cumsum(diff_corrected * diff_volume)
if len(cumulative_area) < len(sorted_volume):

```

```

        cumulative_area = np.append(cumulative_area, cumulative_area[-1] +
                                     diff_corrected[-1] * diff_volume[-1])

ax1.plot(sorted_volume, cumulative_area, label='Integrated Area', color='purple',
         alpha=0.7)

ax1.set_title('Integration and Normalization')
ax1.set_xlabel('Volume (ml)')
ax1.set_ylabel('mAU')
ax1.legend()
ax1.grid(True)

# Bottom row: Bar chart of relative proportions across unique Fraction_unique_IDs
ax2 = axes[1]
fractions = list(relative_proportions1.keys())
proportions = list(relative_proportions1.values())
ax2.bar(fractions, proportions, color='steelblue', edgecolor='black')
ax2.set_title('Relative Proportions Across Fractions (Run 1)')
ax2.set_xlabel('Fraction Unique ID')
ax2.set_ylabel('Relative Proportion')
ax2.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig('/workspace/Integration_and_Normalization.png', dpi=300, bbox_inches='
            tight')
plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import os
from sklearn.impute import KNNImputer

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Check if dataframe is empty
    if df.empty:
        raise ValueError("Input dataframe is empty.")

    # Verify schema: Remove 'Unnamed: 0' if it is an index column
    if 'Unnamed: 0' in df.columns:
        if df['Unnamed: 0'].dtype == 'int64' and df['Unnamed: 0'].is_unique:
            df = df.drop(columns=['Unnamed: 0'])

    # Handle missing values in target columns before calculation
    target_cols = ['UV_1_280_mAU', 'volume_ml']
    imputer = KNNImputer()
    df[target_cols] = imputer.fit_transform(df[target_cols])

    # Define baseline region as first 10% of total run volume for each run
    run_volumes = df.groupby('run_no')['volume_ml'].sum().reset_index()
    run_volumes.columns = ['run_no', 'total_volume_ml']

    # Merge to get baseline cutoff
    df = df.merge(run_volumes, on='run_no')

```

```

df['baseline_cutoff_ml'] = df['total_volume_ml'] * 0.10

# Identify baseline region rows
df_sorted = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

# Calculate cumulative volume to identify the 10% threshold point
df_sorted['cumulative_volume'] = df_sorted.groupby('run_no')['volume_ml'].cumsum()
df_sorted['is_baseline'] = (df_sorted['cumulative_volume'] <= df_sorted.groupby('run_no')['total_volume_ml'] * 0.10)

# Pre-calculate baseline cutoffs and sorted indices outside the loop
# Calculate SNR using vectorized operations
# baseline_std: vectorized std of baseline region per run
baseline_std_per_run = df_sorted.groupby('run_no')['UV_1_280_mAU'].transform(
    lambda x: x[x['is_baseline']].std())

# peak_area_proxy: vectorized sum of non-baseline region per run
peak_area_per_run = df_sorted.groupby('run_no')['UV_1_280_mAU'].transform(
    lambda x: x[~x['is_baseline']].sum())

# Calculate SNR vectorized
df_sorted['SNR'] = peak_area_per_run / baseline_std_per_run

# Identify runs with SNR < 3:1
low_snr_runs = df_sorted[df_sorted['SNR'] < 3]['run_no'].unique()

# Aggregate data by run_no
agg_data = df_sorted.groupby('run_no').agg(
    Total_Fractions=('Fraction_number', 'count'),
    Mean_SNR=('SNR', 'mean'),
    Mean_Rel_Proportion=('Fraction_number', 'mean')
).reset_index()

# Initialize variables to prevent NameError
corr_coef = 0.0
low_snr_runs_list = pd.Series([], dtype=int)
output_text = ''
table_rows = []

# Check for required columns before correlation and raise error if missing
if 'chromatography_stage_order' not in agg_data.columns:
    raise ValueError("Required column 'chromatography_stage_order' is missing from the aggregated data.")
elif len(agg_data) > 1:
    corr_coef = agg_data['chromatography_stage_order'].corr(agg_data['Mean_Rel_Proportion'])
else:
    corr_coef = 0.0

# Prepare Output
findings = []
findings.append(f"Total runs audited: {len(agg_data)}")
findings.append(f"Count of runs with SNR < 3:1: {len(low_snr_runs)}")
findings.append(f"Specific run_no values with SNR < 3:1: {list(low_snr_runs)}")
findings.append(f"Mean SNR per run: {agg_data['Mean_SNR'].mean():.2f}")

output_text = "\n".join(findings)

```

```
# Create Markdown File
output_dir = '/workspace/'
os.makedirs(output_dir, exist_ok=True)

# Construct table content using list comprehension to avoid iterrows overhead
table_rows = [f"|_{row['run_no']}|_|_{row['Total_Fractions']}|_|_{row['Mean_SNR']}|.:.2f}|_|_{row['Mean_Rel_Proportion']}:.2f}|_" for _, row in agg_data.iterrows()]

md_content = f"""# Quality Audit and SNR Calculation Report

## Key Findings
- {output_text}

## Run Summary Table
| run_no | Total Fractions | Mean SNR | Mean Relative Proportion |
|-----|-----|-----|-----|
{ "\n".join(table_rows) }

## Correlation Report
The calculated correlation coefficient between chromatography_stage_order and peak area magnitude (proxied by fraction number) is: {corr_coef:.4f}

"""

output_file = os.path.join(output_dir, 'quality_audit_report.md')
with open(output_file, 'w') as f:
    f.write(md_content)

print(f"Report generated at: {output_file}")
```

Listing 3: Code_3